

팍리스 실무형 일경험 프로그램

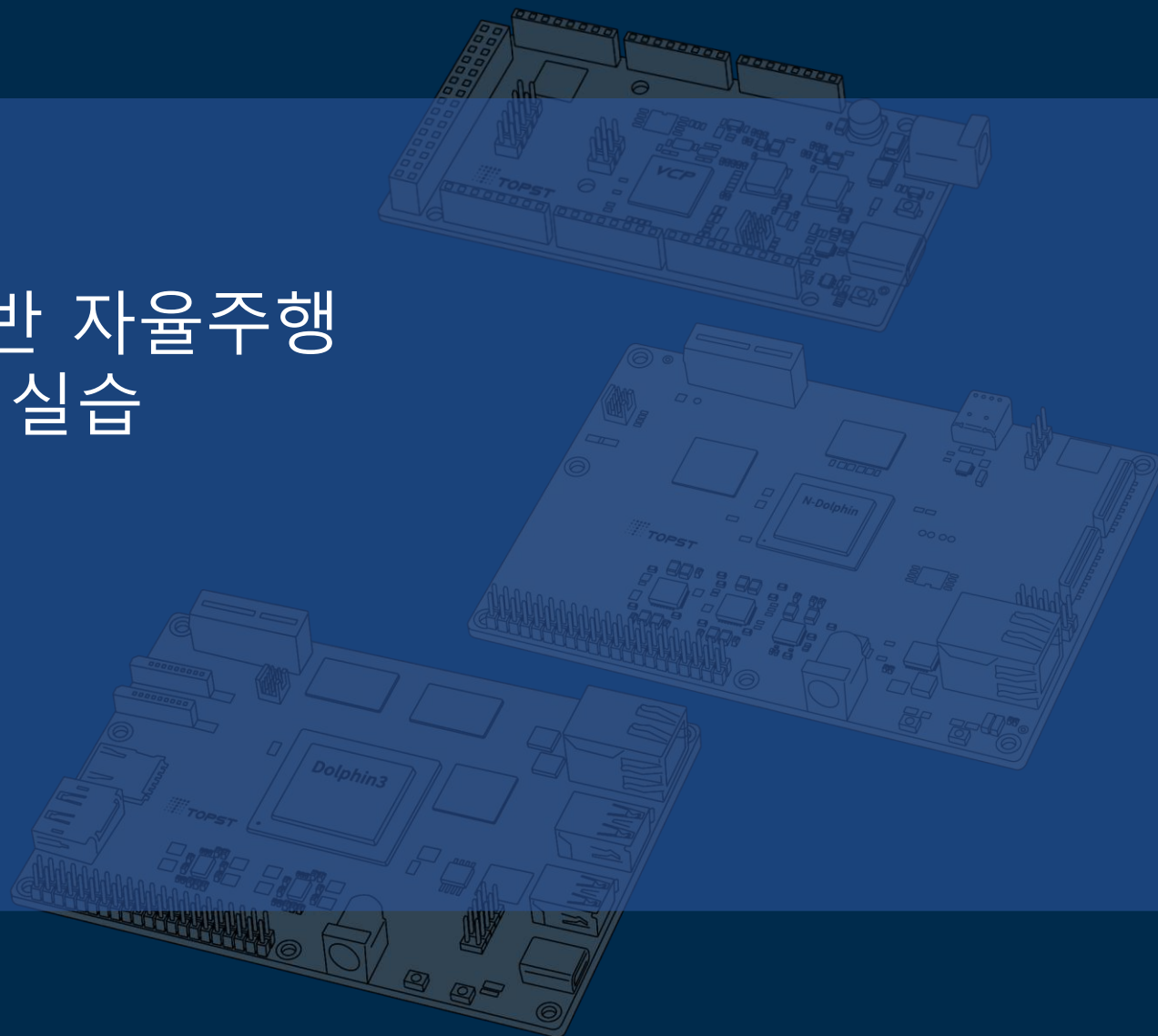
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



5일차

01 ControlZone 구성

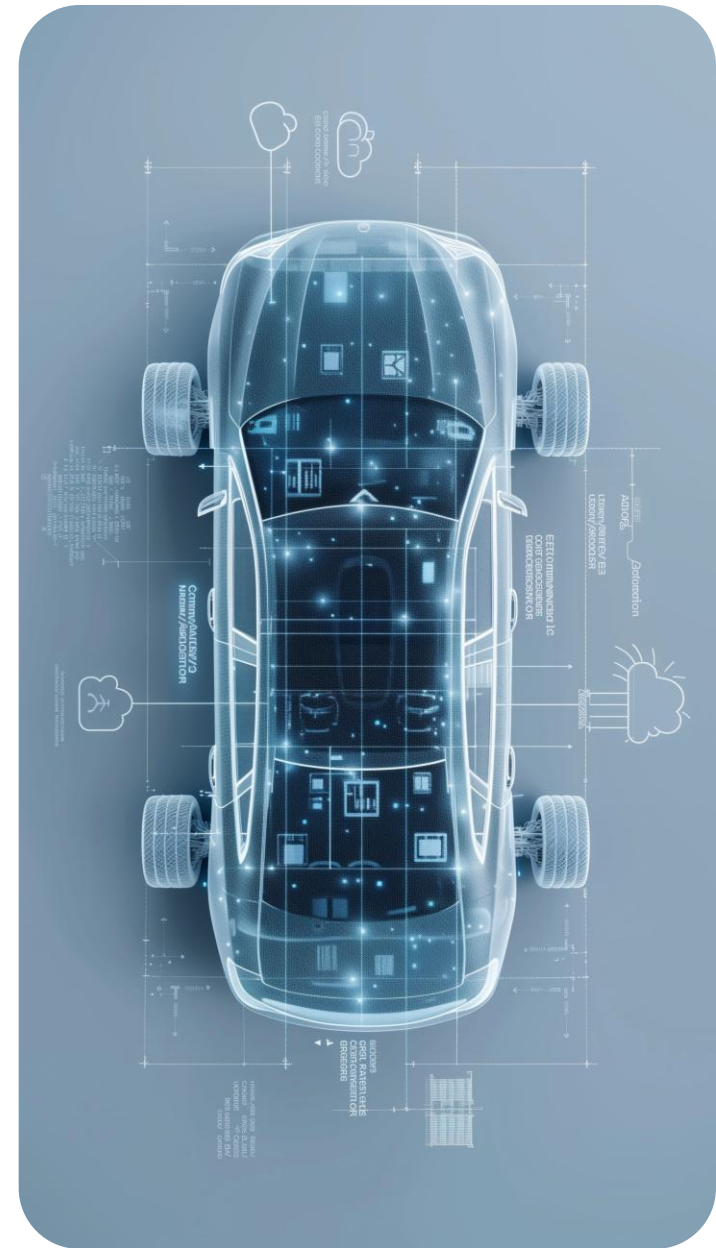
Telechips TOPST



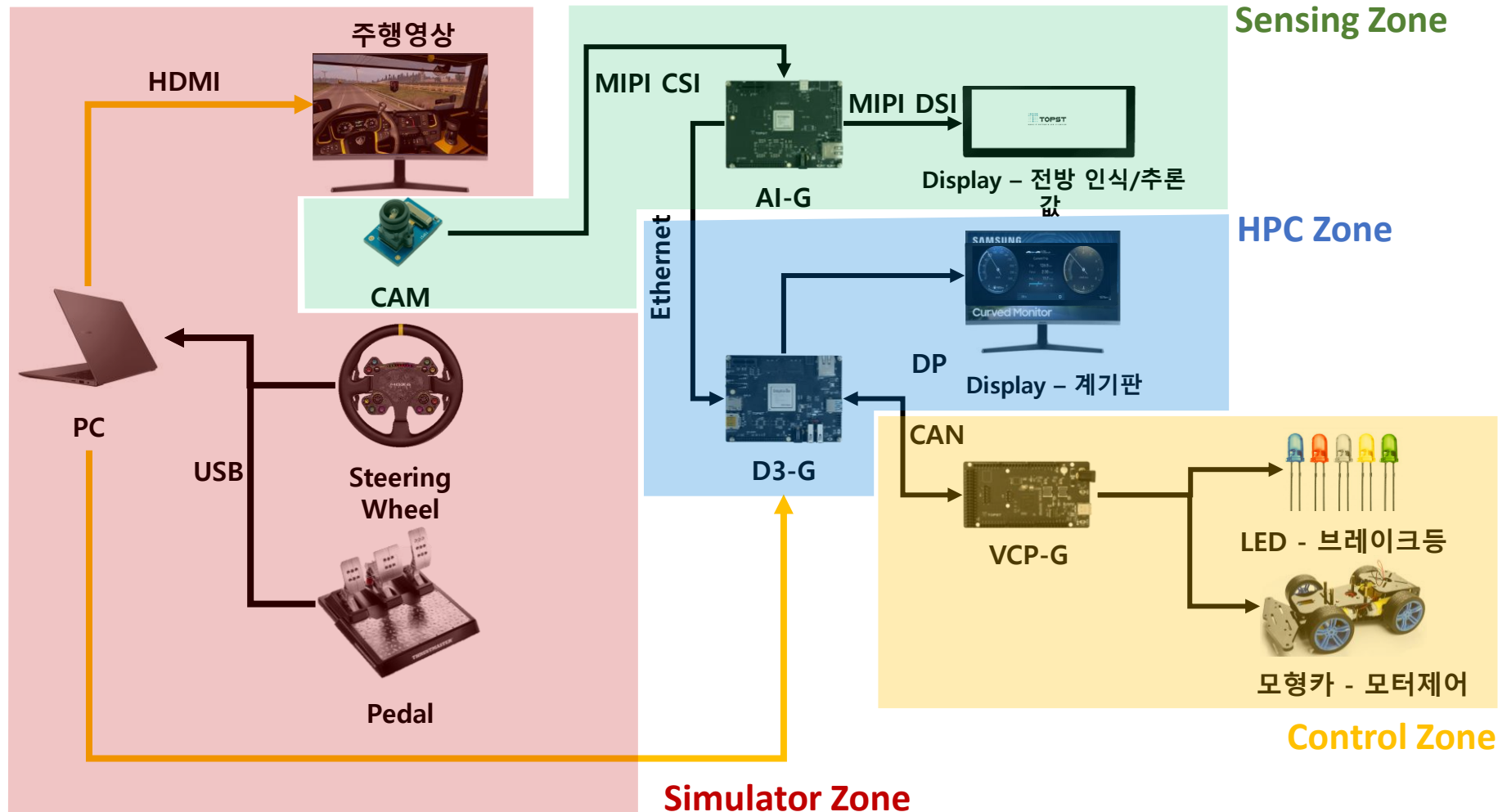
목차

Table of Contents

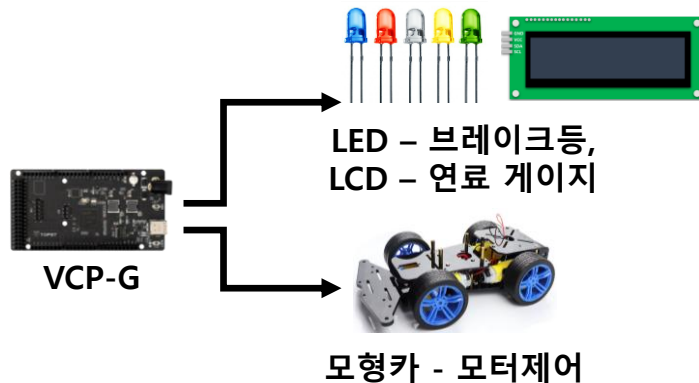
모형차 조립	01
VCP-G, 모형차, Bread Board 연동	02
VCP-G CAN Task에 Control 파트 구현	03
빌드 & FWDN	04



Zonal 기반 차량 인터랙션 시스템-전체 시나리오



Control Zone 구현



Zone Description

VCP-G 의 FreeRTOS 내에
GPIO/I2C/PDM 제어하여 센서
애플리케이션 활용

Task Goal

1. VCP-G 핀맵
2. GPIO Digital out LED
3. PDM DC Motor
4. I2C Display

모형카 조립

아래 페이지 접속 후 pdf파일 다운로드하여 19페이지까지 참고하여 조립,
25페이지는 모터부분만 참고하여 연결

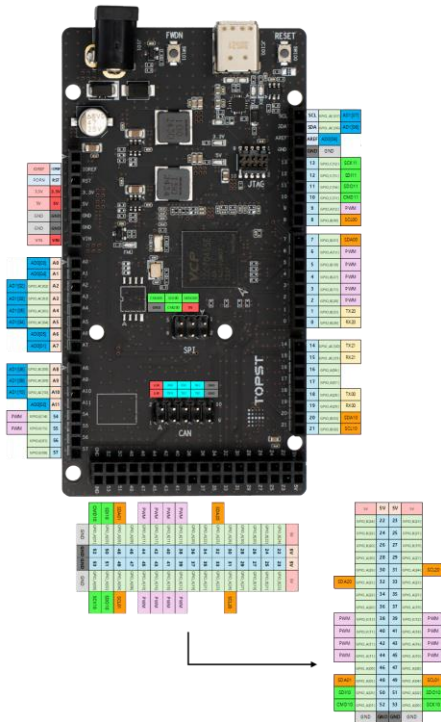
<https://github.com/topst-development/FreeRTOS-VCP/tree/edu/general>

The screenshot shows the GitHub interface for the 'edu/general' branch of the 'topst-development/FreeRTOS-VCP' repository. The branch is 1 commit ahead of 'develop'. The file list includes:

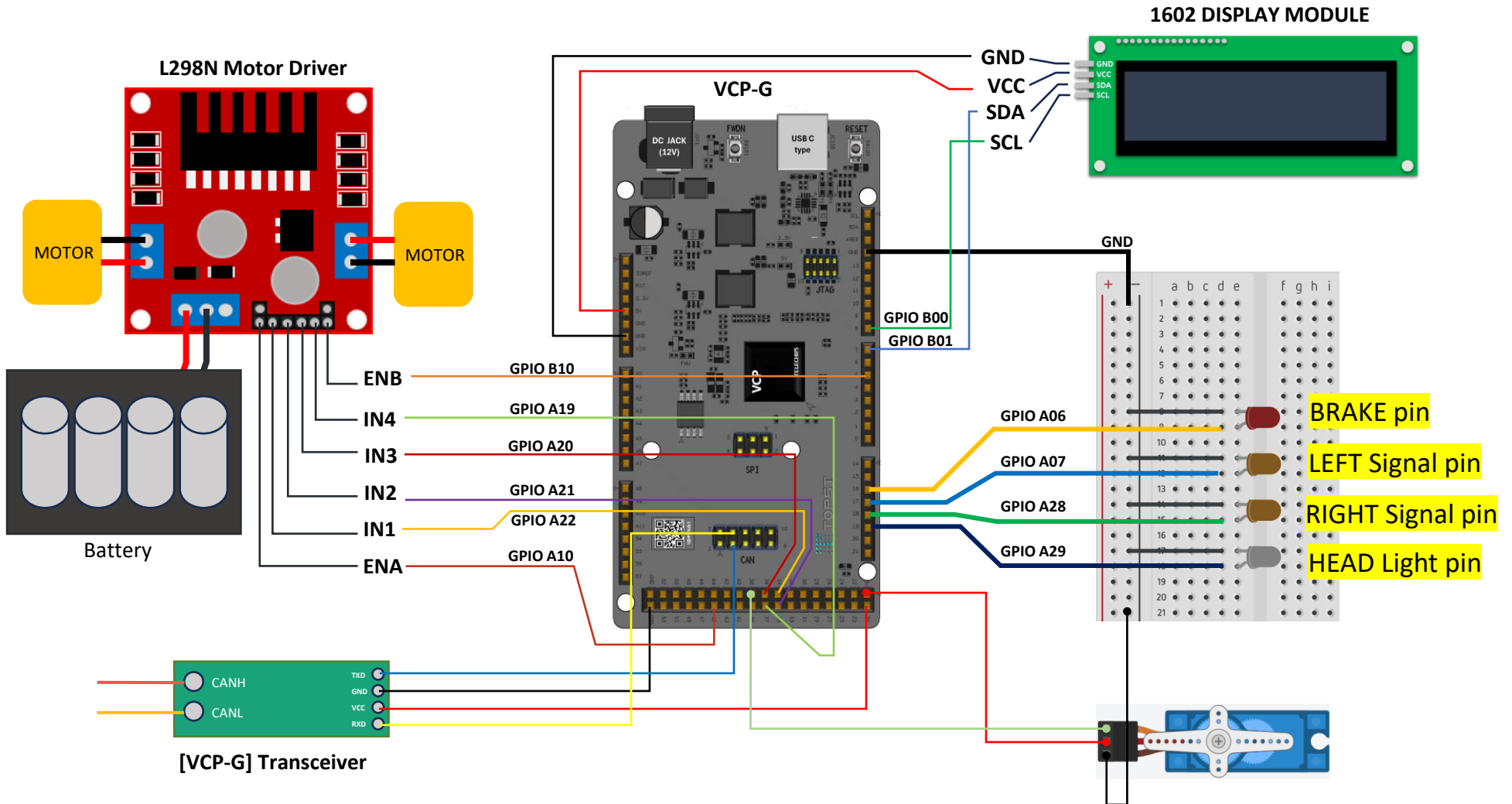
File/Folder	Commit Message	Commit Date
build/tcc70xx	Initial Commit	5 months ago
scripts/debug	Initial Commit	5 months ago
sources	Initial Commit	5 months ago
tools	Initial Commit	5 months ago
LICENSE	Initial Commit	5 months ago
easy-setup_vcp-g.sh	Initial Commit	5 months ago
모형카 조립.pdf	add Assembling a model car pdf file	1 minute ago

VCP-G 핀맵

- TOPST DOCS에서 VCP-G 핀 FUNC 및 GPIO 번호 확인
 - <https://topst.ai/tech/docs?page=1714a3dcf19dbf61088d7364d61e29f90a15ff2>



VCP-G 와 모형차 Kit 회로 연결



소스코드 다운로드

- **FreeRTOS 소스코드 다운로드**

- FreeRTOS Repository edu 폴더로 클론
 - `$ git clone https://github.com/topst-development/FreeRTOS-VCP.git edu`
- edu 폴더로 이동
 - `$ cd edu`
- edu/general 브랜치로 체크아웃
 - `$ git checkout edu/general`
- `./easy-setup_vcp-g.sh` 실행하여 라이선스 동의

Can Control 제어(3) – can_vcp_ctrl.h

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.h

```
#ifndef MCU_BSP_CAN_VCP_CTRL_HEADER
#define MCU_BSP_CAN_VCP_CTRL_HEADER

#if ( MCU_BSP_SUPPORT_CAN_DEMO == 1 ) // CAN 데모 기능이 활성화된 경우에만 포함

#define BrakeLEDPIN      GPIO_GPA(6) // 브레이크등 LED 핀
#define LeftSignalledPIN GPIO_GPA(7) // 좌측 방향등 LED 핀
#define RightSignalledPIN GPIO_GPA(28) // 우측 방향등 LED 핀
#define HeadLEDPIN       GPIO_GPA(29) // 전조등 LED 핀

#define IN1  GPIO_GPA(22) // 모터 제어 입력 1
#define IN2  GPIO_GPA(21) // 모터 제어 입력 2
#define IN3  GPIO_GPA(20) // 모터 제어 입력 3
#define IN4  GPIO_GPA(19) // 모터 제어 입력 4

#define ENA_SEL      0 // 모터 A PDM 채널 선택
#define ENA_PORT      GPIO_PERICH_CH0 // 모터 A PDM 포트
#define ENB_SEL      4 // 모터 B PDM 채널 선택
#define ENB_PORT      GPIO_PERICH_CH1 // 모터 B PDM 포트

#define PDM_PERIOD_NS (250000) // PDM 주기
#define SPEED_MAX      80 // 최대 속도 값
#define DUTY_MAX      80 // 최대 듀티 비
#define MIN_ON_NS      15000 // 최초 ON 시간

#endif
```

Can Control 제어(3) – can_vcp_ctrl.h

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.h

```
#define I2C_CH      0    // I2C 채널, 포트, LCD 주소, 통신 속도, LCD 명령 전송 모드, LCD 데이터 전송 모드 설정
#define I2C_PORT    0
#define LCD_ADDR    0x27
#define I2C_SPEED   100    // kHz
#define LCD_CMD     0
#define LCD_DATA    1

enum VCP_IO_TYPE {    // CAN 메시지 ID에 대응되는 제어 타입 정의
    VCP_IO_BREAK_LIGHT=0x101,
    VCP_IO_TURN_SIGNAL=0x102,
    VCP_IO_EMER_SIGNAL=0x103,
    VCP_IO_HEAD_LIGHT=0x104,
    VCP_IO_FUEL_LEVEL=0x105,
    VCP_IO_MOTOR_SPEED=0x106,
    VCP_IO_MOTOR_WHEEL=0x107
};

enum VCP_IO_ACTION {    // ON/OFF 상태 정의
    VCP_IO_ACTION_ON=0x01,
    VCP_IO_ACTION_OFF=0x02
};

enum VCP_IO_SUBTYPE {    // 세부 방향 정의
    VCP_IO_SUB_LEFT=0x01,
    VCP_IO_SUB_RIGHT=0x02
};
```

Can Control 제어(3) – can_vcp_ctrl.h

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.h

```
void ControlBreadBoardSensors(uint32 mId, uint8 nDataLength, sint8* pucData); // CAN 메시지 기반 센서 제어 함수
boolean InitSensorControls(void); // 센서 초기화 함수

#endif // ( MCU_BSP_CAN_VCP_CTRL_HEADER == 1 )

#endif // MCU_BSP_CAN_DEMO_HEADER
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
#if ( MCU_BSP_SUPPORT_CAN_DEMO == 1 ) // 활성화 된 경우에만 이 파일 컴파일

#include <app_cfg.h>

#include "bsp.h"
#include "gic.h"
#include "gpio.h"
#include "debug.h"
#include "pdm.h"
#include "i2c.h"
#include "stdio.h"

#include "can_vcp_ctrl.h"
#define MIN_DUTY      (35) // PDM 최소 듀티 비(%)

static PDMModeConfig_t cfgA, cfgB; // 두 개의 모터 설정 구조체 저장용

static void BrakeLED_ON(void); // 브레이크등 제어 함수 프로토타입
static void BrakeLED_OFF(void);
static void LeftSignalLED_ON(void); // 방향등(좌/우) LED 제어 함수 프로토타입
static void LeftSignalLED_OFF(void);
static void RightSignalLED_ON(void);
static void RightSignalLED_OFF(void);
static void HeadLED_ON(void); // 전조등 LED 제어 함수 프로토타입
static void HeadLED_OFF(void);
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
static void lcd_send(uint8 mode, uint8 data); // LCD 제어용 I2C 전송 관련 함수
static void lcd_cmd(uint8 cmd);
static void lcd_data(uint8 dat);
static void lcd_init(void);
static void lcd_print(const char *str);

static void pin_out(uint32 p); // GPIO 출력 유틸리티 함수
static void pin_hi (uint32 p);
static void pin_lo (uint32 p);

static int pdm_apply(uint32 ch, PDMModeConfig_t* cfg); // 모터용 PDM 제어 내부 함수
static void MotorPDM_Init(void);
static void MotorA_Set(uint32 duty_pct, uint32 forward);
static void MotorB_Set(uint32 duty_pct, uint32 forward);
static uint32 duty_pct_to_ns(uint32 pct, uint32 period_ns);
static sint8 duty_from_speed(sint8 speed);

static void ControlBrakeLight(boolean bTurnOn); // 센서 제어 함수
static void ControlSignalLight(boolean bLeft, boolean bTurnOn);
static void ControlHeadLight(boolean bTurnOn);
static void ControlFuelLevel(uint8 fuelLevel);
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
boolean InitSensorControls(void) // 모터 PDM, I2C, LCD 초기화 함수
{
    MotorPDM_Init();
    I2C_Init();
    if (I2C_Open(I2C_CH, I2C_PORT, I2C_SPEED, NULL_PTR, NULL_PTR) != SAL_RET_SUCCESS)
    {
        mcu_printf("[LCD] I2C_Open failed (ch=%d, port=%d, speed=%d)\r\n",
                    (int)I2C_CH, (int)I2C_PORT, (int)I2C_SPEED);
        return FALSE;
    }
    I2C_ScanSlave(I2C_CH);
    lcd_init();
    return TRUE;
}

void BrakeLED_ON(void)
{
    GPIO_Set(BrakeLEDPIN, 1); // 브레이크등 ON
}

void BrakeLED_OFF(void)
{
    GPIO_Set(BrakeLEDPIN, 0); // 브레이크등 OFF
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
void ControlBrakeLight(boolean bTurnOn)
{
    static boolean binit = FALSE;  // 핀 초기화 여부

    mcu_printf("[BRAKE] Controlling Brake Light\r\n");
    if (binit == FALSE) {  // 아직 초기화 안 된 경우
        GPIO_Config(BrakeLEDPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) | GPIO_INPUTBUF_DIS);
        BrakeLED_OFF();  // 초기 상태 OFF
        binit = TRUE;
    }
    // 브레이크등 ON/OFF
    if (bTurnOn) {
        BrakeLED_ON();
        mcu_printf("[BRAKE] ON\r\n");
    } else {
        BrakeLED_OFF();
        mcu_printf("[BRAKE] OFF\r\n");
    }
}

void LeftSignalLED_ON(void)
{
    GPIO_Set(LeftSignalLEDPIN, 1);  // 좌측 방향등 ON
}

void LeftSignalLED_OFF(void)
{
    GPIO_Set(LeftSignalLEDPIN, 0);  // 좌측 방향등 OFF
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
void RightSignalled_ON(void)
{
    GPIO_Set(RightSignalledPIN, 1); // 우측 방향등 ON
}

void RightSignalled_OFF(void)
{
    GPIO_Set(RightSignalledPIN, 0); // 우측 방향등 OFF
}

void ControlSignalLight(boolean bLeft, boolean bTurnOn)
{
    static boolean binit = FALSE;

    mcu_printf("[SIGNAL] Controlling Signal Light\r\n");
    if(binit == FALSE) {
        // 좌/우 방향등 GPIO 설정
        GPIO_Config(LeftSignalledPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) |
GPIO_INPUTBUF_DIS);
        GPIO_Config(RightSignalledPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) |
GPIO_INPUTBUF_DIS);
        LeftSignalled_OFF();
        RightSignalled_OFF();
        binit = TRUE;
    }
}
```


Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
// 좌/우 방향으로 제어
if (bLeft) {
    if (bTurnOn) {
        LeftSignalLED_ON();
        mcu_printf("[LeftSignal] ON\r\n");
    }
    else {
        LeftSignalLED_OFF();
        mcu_printf("[LeftSignal] OFF\r\n");
    }
} else {
    if (bTurnOn) {
        RightSignalLED_ON();
        mcu_printf("[RightSignal] ON\r\n");
    }
    else {
        RightSignalLED_OFF();
        mcu_printf("[RightSignal] OFF\r\n");
    }
}
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
void HeadLED_ON(void)
{
    GPIO_Set(HeadLEDPIN, 1); // 전조등 ON
}

void HeadLED_OFF(void)
{
    GPIO_Set(HeadLEDPIN, 0); // 전조등 OFF
}

void ControlHeadLight(boolean bTurnOn)
{
    static boolean binit = FALSE; // 핀 초기화 여부

    mcu_printf("[LIGHT] Controlling Head Light\r\n");
    if (binit == FALSE) { // 아직 초기화 안 된 경우
        GPIO_Config(HeadLEDPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) | GPIO_INPUTBUF_DIS);
        HeadLED_OFF(); // 전조등 OFF
        binit = TRUE;
    }

    if (bTurnOn) {
        HeadLED_ON();
        mcu_printf("[Head Light] ON\r\n");
    } else {
        HeadLED_OFF();
        mcu_printf("[Head Light] OFF\r\n");
    }
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
void lcd_send(uint8 mode, uint8 data)
{
    uint8 high = data & 0xF0; // 상위 4비트
    uint8 low  = (data << 4) & 0xF0; // 하위 4비트
    uint8 buf[6];

    // LCD 제어 비트
    buf[0] = high | 0x08 | (mode ? 0x01 : 0x00);
    buf[1] = high | 0x0C | (mode ? 0x01 : 0x00);
    buf[2] = high | 0x08 | (mode ? 0x01 : 0x00);
    buf[3] = low  | 0x08 | (mode ? 0x01 : 0x00);
    buf[4] = low  | 0x0C | (mode ? 0x01 : 0x00);
    buf[5] = low  | 0x08 | (mode ? 0x01 : 0x00);

    I2CXfer_t xfer = {
        .xCmdLen = 0,
        .xOutLen = 6,
        .xOutBuf = buf,
        .xInLen  = 0,
        .xInBuf  = NULL,
        .xCmdBuf = NULL,
        .xOpt    = 0
    };

    (void)I2C_Xfer(I2C_CH, (uint8)(LCD_ADDR << 1), xfer, 0); // I2C 전송
    SAL_TaskSleep(2);
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
void lcd_cmd(uint8 cmd)
{
    lcd_send(LCD_CMD, cmd); // LCD 제어 명령어 전송
}

void lcd_data(uint8 dat)
{
    lcd_send(LCD_DATA, dat); // 데이터 전송
}

void lcd_init(void)
{
    SAL_TaskSleep(50);
    lcd_cmd(0x33);
    lcd_cmd(0x32);
    lcd_cmd(0x28);
    lcd_cmd(0x0C);
    lcd_cmd(0x06);
    lcd_cmd(0x01);
    SAL_TaskSleep(5);
}

void lcd_print(const char *str)
{
    while (*str) lcd_data((uint8)*str++); // 문자열을 한 글자씩 LCD에 전송
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
void ControlFuelLevel(uint8 fuelLevel)
{
    static uint8 prev      = 0xFF;  // 이전 값 저장 (불필요한 재출력 방지)

    uint8 pct = (fuelLevel > 100) ? 100 : fuelLevel;  // 범위 0~100%로 제한

    if (pct == prev)  // 변화 없으면 종료
    {
        return;
    }
    prev = pct;

    lcd_cmd((uint8)(0x80 | 0x00));  // 첫 번째 줄
    lcd_print("Fuel Level      ");

    char line[17];
    (void)snprintf(line, sizeof(line), "Fuel: %3u%%      ", (unsigned)pct);
    lcd_cmd((uint8)(0x80 | 0x40));  // 두 번째 줄
    lcd_print(line);

    mcu_printf("[FUEL] Controlling Fuel Level\r\n");
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
// GPIO 출력 설정
static inline void pin_out(uint32 p) { GPIO_Config(p, GPIO_OUTPUT | GPIO_FUNC(0) | GPIO_DS(3)); }
static inline void pin_hi(uint32 p) { GPIO_Set(p, 1); }
static inline void pin_lo(uint32 p) { GPIO_Set(p, 0); }
static int pdm_apply(uint32 ch, PDMModeConfig_t* cfg) // PDM 설정 적용 함수
{
    (void)PDM_Disable(ch, PMM_OFF); // 채널 비활성화
    uint32 wait = 0;

    while (PDM_GetChannelStatus(ch) && wait < 100) {
        SAL_TaskSleep(1);
        wait++;
    }

    if (PDM_SetConfig(ch, cfg) != SAL_RET_SUCCESS) return -1; // 설정 적용
    if (PDM_Enable(ch, PMM_OFF) != SAL_RET_SUCCESS) return -2; // 채널 활성화
    return 0;
}

// 듀티 비율(%) -> 나노초 단위 변환
uint32 duty_pct_to_ns(uint32 pct, uint32 period_ns)
{
    if (pct == 0) return 0;
    if (pct > 100) pct = 100;
    if (pct < MIN_DUTY && pct > 0) pct = MIN_DUTY;
    uint64 num = (uint64)period_ns * (uint64)pct + 50ULL;
    return (uint32)(num / 100ULL);
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
sint8 duty_from_speed(sint8 speed) // 속도값(0~80) -> 듀티비 변환
{
    if (speed == 0)
    {
        return 0;
    }
    if (speed > 80)
    {
        speed = 80;
    }
    if (speed < 0)
    {
        return 0;
    }

    return 40 + ((speed - 1) * 80) / 79;
}

void MotorPDM_Init(void) // PDM 초기화 함수
{
    static boolean initd = FALSE;
    if (initd) return;

    // GPIO 초기화
    pin_out(IN1); pin_out(IN2);
    pin_out(IN3); pin_out(IN4);
    pin_lo(IN1); pin_lo(IN2);
    pin_lo(IN3); pin_lo(IN4);
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
// PDM 초기화
PDM_Init();
PDM_CfgSetWrPw();
PDM_CfgSetWrLock(0);
// 모터 A 설정
SAL_MemSet(&cfgA, 0, sizeof(cfgA));
cfgA.mcPortNumber      = ENA_PORT;
cfgA.mcOperationMode   = PDM_OUTPUT_MODE_PHASE_1;
cfgA.mcPeriodNanoSec1  = PDM_PERIOD_NS;
cfgA.mcDutyNanoSec1    = 0;
(void)pdm_apply(ENA_SEL, &cfgA);
// 모터 B 설정
SAL_MemSet(&cfgB, 0, sizeof(cfgB));
cfgB.mcPortNumber      = ENB_PORT;
cfgB.mcOperationMode   = PDM_OUTPUT_MODE_PHASE_1;
cfgB.mcPeriodNanoSec1  = PDM_PERIOD_NS;
cfgB.mcDutyNanoSec1    = 0;
(void)pdm_apply(ENB_SEL, &cfgB);

initd = TRUE;
}
```


Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
void MotorA_Set(uint32 duty_pct, uint32 forward) // 모터 A 제어
{
    if (duty_pct == 0) {
        pin_lo(IN1); pin_lo(IN2); // 정지
    } else if (forward) {
        pin_lo(IN1); pin_hi(IN2); // 정방향
    } else {
        pin_hi(IN1); pin_lo(IN2); // 역방향
    }

    cfgA.mcDutyNanoSec1 = duty_pct_to_ns(duty_pct, cfgA.mcPeriodNanoSec1);
    (void)pdm_apply(ENA_SEL, &cfgA);
}

void MotorB_Set(uint32 duty_pct, uint32 forward) // 모터 B 제어
{
    if (duty_pct == 0) {
        pin_lo(IN3); pin_lo(IN4); // 정지
    } else if (forward) {
        pin_hi(IN3); pin_lo(IN4); // 정방향
    } else {
        pin_lo(IN3); pin_hi(IN4); // 역방향
    }

    cfgB.mcDutyNanoSec1 = duty_pct_to_ns(duty_pct, cfgB.mcPeriodNanoSec1);
    (void)pdm_apply(ENB_SEL, &cfgB);
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
static void ConfigureServoPDM(uint32 channel, uint32 port, uint32 angle_deg) // 서보모터 설정 함수
{
    PDMModeConfig_t pwm_cfg;
    uint32 duty_ns = 500000 + (angle_deg * (2000000 / 180)); // 0~180도 → 0.5~2.5ms
    uint32 wait_cnt = 0;

    pwm_cfg.mcPortNumber      = port;
    pwm_cfg.mcOperationMode   = PDM_OUTPUT_MODE_PHASE_1;
    pwm_cfg.mcInversedSignal  = 0;
    pwm_cfg.mcOutSignalInIdle = 0;
    pwm_cfg.mcLoopCount       = 0;
    pwm_cfg.mcOutputCtrl      = 0;

    pwm_cfg.mcPeriodNanoSec1 = 2000000; // 20ms (50Hz)
    pwm_cfg.mcDutyNanoSec1   = duty_ns;
    pwm_cfg.mcPeriodNanoSec2 = 0;
    pwm_cfg.mcDutyNanoSec2   = 0;

    PDM_Disable(channel, PMM_ON); // PDM 재적용
    while (PDM_GetChannelStatus(channel))
    {
        SAL_TaskSleep(1);
        if (++wait_cnt > 100)
        {
            mcu_printf("Timeout on channel %d\n", channel);
            return;
        }
    }
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
if (PDM_SetConfig(channel, &pwm_cfg) != SAL_RET_SUCCESS)
{
    mcu_printf("SetConfig fail (CH:%d)\n", channel);
    return;
}

if (PDM_Enable(channel, PMM_ON) != SAL_RET_SUCCESS)
{
    mcu_printf("Enable fail (CH:%d)\n", channel);
    return;
}

mcu_printf("CH%d angle: %3d° → duty: %d ns\n", channel, angle_deg, duty_ns);
}
void ControlBreadBoardSensors(uint32 mId, uint8 nDataLength, sint8* pucData) // CAN 메시지 처리 함수
{
    sint8 ucMsgData[2] = {0x00, 0x00};

    if(nDataLength == 0)
    {
        return; // 데이터 없으면 무시
    }

    switch(mId)
    {
        case VCP_IO_BREAK_LIGHT: // 브레이크등
            ucMsgData[0] = pucData[0];
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
        if(ucMsgData[0] == VCP_IO_ACTION_ON)
        {
            ControlBrakeLight(TRUE);
        }
        else
        {
            ControlBrakeLight(FALSE);
        }

        break;
case VCP_IO_TURN_SIGNAL: // 방향등
    ucMsgData[0] = pucData[0];
    ucMsgData[1] = pucData[1];

    if (ucMsgData[0] == VCP_IO_SUB_LEFT)
    {
        if(ucMsgData[1] == VCP_IO_ACTION_ON)
        {
            // turn on the left signal
            ControlSignalLight(TRUE, TRUE);
        }
        else
        {
            // turn off the left light
            ControlSignalLight(TRUE, FALSE);
        }
    }
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
        if (ucMsgData[0] == VCP_IO_SUB_RIGHT)
        {
            if(ucMsgData[1] == VCP_IO_ACTION_ON)
            {
                ControlSignalLight(FALSE, TRUE);
            }
            else
            {
                ControlSignalLight(FALSE, FALSE);
            }
        }

        break;

case VCP_IO_EMER_SIGNAL: // 비상등
    ucMsgData[0] = pucData[0];
    if(ucMsgData[0] == VCP_IO_ACTION_ON)
    {
        // turn on the emergency signal lights
        ControlSignalLight(TRUE, TRUE);
        ControlSignalLight(FALSE, TRUE);
    }
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
        else
        {
            ControlSignalLight(TRUE, FALSE);
            ControlSignalLight(FALSE, FALSE);
        }

        break;

case VCP_IO_HEAD_LIGHT: // 전방 라이트
    ucMsgData[0] = pucData[0];
    if(ucMsgData[0] == VCP_IO_ACTION_ON)
    {
        ControlHeadLight(TRUE);
    }
    else
    {
        ControlHeadLight(FALSE);
    }

    break;
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
case VCP_IO_FUEL_LEVEL: // 연료게이지
    ucMsgData[0] = (pucData[0] > 100)?100:pucData[0];
    ControlFuelLevel(ucMsgData[0]);
    break;
case VCP_IO_MOTOR_SPEED: // 모터 스피드
{
    static sint8 duty = 0;
    sint8 speed = pucData[0]; // 0~80

    if(speed==0)
    {
        MotorA_Set(0, 1);
        MotorB_Set(0, 1);
    }
    else if(speed > 0)
    {
        duty = duty_from_speed(speed);
        MotorA_Set(duty, 1);
        MotorB_Set(duty, 1);
    }
    else
    {
        duty = duty_from_speed(speed);
        MotorA_Set(duty, 0);
        MotorB_Set(duty, 0);
    }
}
```

Can Control 제어(3) – can_vcp_ctrl.c

~/edu/sources/app.sample/app.can.demo/can_vcp_ctrl.c

```
        mcu_printf("[MOTOR] speed=%d => duty=%d%%\r\n", (sint8)speed, (sint8)duty);

        break;
    }
    case VCP_IO_MOTOR_WHEEL: // 서보모터
    {
        int8_t input = (int8_t)pucData[0]; // 0~256
        if (input < 0) input = 0;
        if (input > 127) input = 127;

        // 중앙 기준으로 -45~+45도로 매핑
        int angle = 90 + ((input - 65) * 45) / 65;

        ConfigureServoPDM(5, GPIO_PERICH_CH3, angle);

        mcu_printf("[SERVO] input=%d -> angle=%d deg\r\n", input, angle);
        break;
    }
    default:
        mcu_printf("[%s][%d] undefined can id !!!\r\n", __FUNCTION__, __LINE__);
        break;
}

}
#endif
```

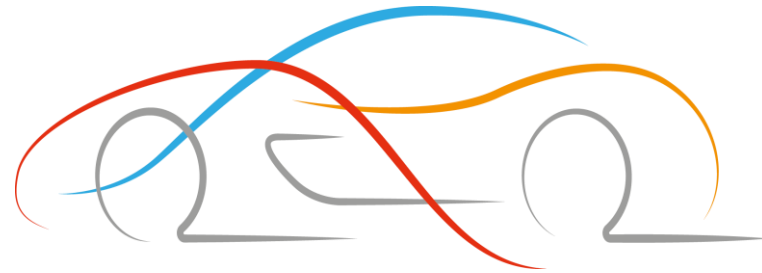

GPIO Code – rules.mk / main.c

~/edu/sources/app.sample/app.can.demo/rules.mk

```
SRCS += can_demo.c  
//추가  
SRCS += can_vcp_ctrl.c
```

빌드 & FWDN

- build 진행
 - `$ cd ~/edu/build/tcc70xx/gcc`
 - `$ make clean && make`
- Build 완료 후 FWDN (빌드한 디렉토리에서 바로 실행)
 - `$ sudo ~/edu/tools/fwdn_vcp/fwdn`
`--fwdn ~/edu/tools/fwdn_vcp/vcp_fwdn.rom`
`-w ~/edu/build/tcc70xx/gcc/output/tcc70xx_pflash_boot_2M_ECC.rom`
- CAN 메시지 listening, 센서 제어 항상 대기



Thank You